

《编译原理》 Lab 2 : 语义分析

姓名 / 学号 : 林武文 / 3230300081 专业 : 计算机科学与技术

日期 : 23/4/2026

Lab Introduction

Implementing **semantic analysis** which construct and manage `symbol table` + perform `Type Checking`, report semantic errors + correct exit codes

Lab Approach / 试验思路

1. Type System : `type.hpp` & `type.cpp`

Defines **type system** used during semantic analysis

→ Implemented using base class `Type` and derived classes (`PrimitiveType`, `ArrayType`, `FuncType`)

e.g. : Using `compatible()` to match array type compatibility rule

```
bool ArrayType::compatible(const TypePtr& other) const {
    auto other_type = std::dynamic_pointer_cast<ArrayType>(other);
    if (!other_type) return false;
    return element_type->compatible(other_type->element_type) &&
           dims == other_type->dims;}
```

→ Checks whether current array type is compatible with " `other` " type (`boolean`)

1. Casting `other` to array (ensuring array-to-array comparison) + Compare

◦ If `other` isn't an array → not compatible (`false`)

2. Then recursively checks whether element types are compatible (for nested arrays)

→ Then checks whether the dimensions of the array is the same

2. Symbol Table : `symbol_table.hpp` & `symbol_table.cpp`

Manages identifiers & scope, as **stack of hash maps** (each map as 1 scope)

```
std::vector<std::unordered_map<std::string, SymbolPtr>> scope_stack;
```

```
SymbolPtr SymbolTable::add_symbol(std::string name, TypePtr type) {  
    if (find_symbol(name, true)) return nullptr; // e.g.: symbol insertion  
    auto symbol = Symbol::create(name, type);  
    if (scope_stack.size() <= 1)  
        symbol->unique_name = name;  
    else  
        symbol->unique_name = name + "_" + std::to_string(next_unique_id++);  
    scope_stack.back()[name] = symbol;  
    return symbol;}
```

- Ensures no redefinition in the same scope + unique variable handler (e.g. `x_0`) + Inserts new symbol into hash map of current scope + return created symbol

3. AST Extension : `tree.hpp`

Support semantic analysis, AST nodes were extended with pointer to symbol

- After semantic resolution, each node then can reference its symbol table entry
 - e.g. : `FuncCall` node stores which function is being called

4. Type Checker : `type_checker.cpp` & `type_checker.hpp`

`TypeChecker` performs **semantic analysis** by traversing AST recursively.

1. Dispatch : determines AST kind + calls the corresponding checking function

- e.g. : `CHECK_NODE(FuncDef)` , `CHECK_NODE(VarDecl)`

2. Function Def. Check : ensure no redefining, param & body checked in new scope

```
auto existing = symbol_table.find_symbol(node->name, true);  
if (existing) {  
    std::exit(11);} // redefinition  
node->symbol = symbol_table.add_symbol(node->name, func_type);  
symbol_table.enter_scope(); // the rest is SNIPPED
```

3. Variable Def & Init : enforcing correct type + correct array initialization structure

```

if (node->init) {
    if (!node->dims.empty()) {
        validate_array_init(init_val, node->dims);} else {
        TypePtr init_type = check(node->init);
        if (!type->compatible(init_type)) { std::exit(4); // SNIPPED

```

4. **Array Init** : ensuring nested init are valid + element count doesn't exceed capacity

```

int TypeChecker::validate_array_init(AST::InitValPtr init,
                                     const std::vector<int>& dims) {
    int total = array_capacity(dims); int used = 0
    for (auto& elem : init->elements) { // SNIPP
        if (used > total) { std::exit(3); // excess elements, SNIPPING

```

5. **Function Call Checking** : match dimensions → special handling for array params
6. **LVal and array indexing** ensures variables declared first, only arrays can be indexed, index must be `int`

Difficulties and Highlights / 难点与亮点

Difficulties : multi-dimension Array init (`Excess Elements`)

- Initially counting the number of elements + compare it with total array capacity ❌

→ Validate recursively : by recursively traversing `InitVal` + Match nested list with correct sub-array boundaries + track the consumed elements + excess error

Highlights : Semantic Checking Flow based on AST tree traversal

Semantic analysis organized with `check()` function that dispatch each AST node to the corresponding handler (`checkLVal`, ...). Each handler does local semantic validation + return types, propagates bottom-up type. AST structure determines evaluation order.

Reflection / 心得体会

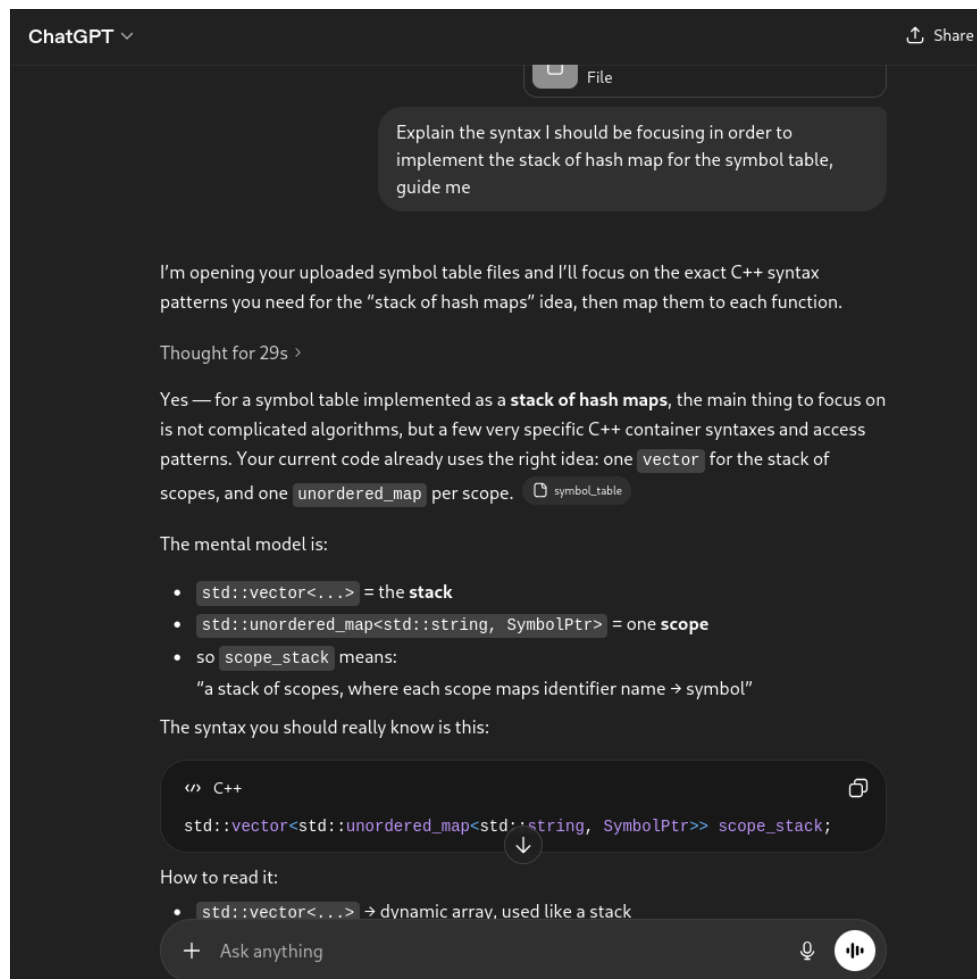
This lab implements CH.5 semantic analysis involving type system, symbol table, AST type checking. Through scope and recursive validation ensure correctness for expressions, function calls, array init. For the report, due to the strict page limit, many important implementation details had to be compressed or omitted forcefully, including difficulties & highlights which needs more explanations especially on array initialization and type compatibility could not be fully explained.

Additional Content / 附加内容

借鉴声明 : I didn't have any reference while doing this lab

AI 工具使用记录 : I only use ChatGPT (GenAI) for assist during the lab

For each code, the first step is to ask the basic syntax and guidance on key files



- While it also assists in explaining the core concepts for each file along with syntax explanation

4. Syntax for searching inside one hash map

In your code:

```
auto it = scope_stack.back().find(name);  
if (it != scope_stack.back().end()) return it->second;
```

This is standard `unordered_map` lookup syntax.

Focus on these parts:

`find(name)`

Returns an iterator to the key if found.

`end()`

Special iterator meaning "not found".

`it->second`

If `it` points to a key-value pair:

- `it->first` = key
- `it->second` = value

So this pattern means:

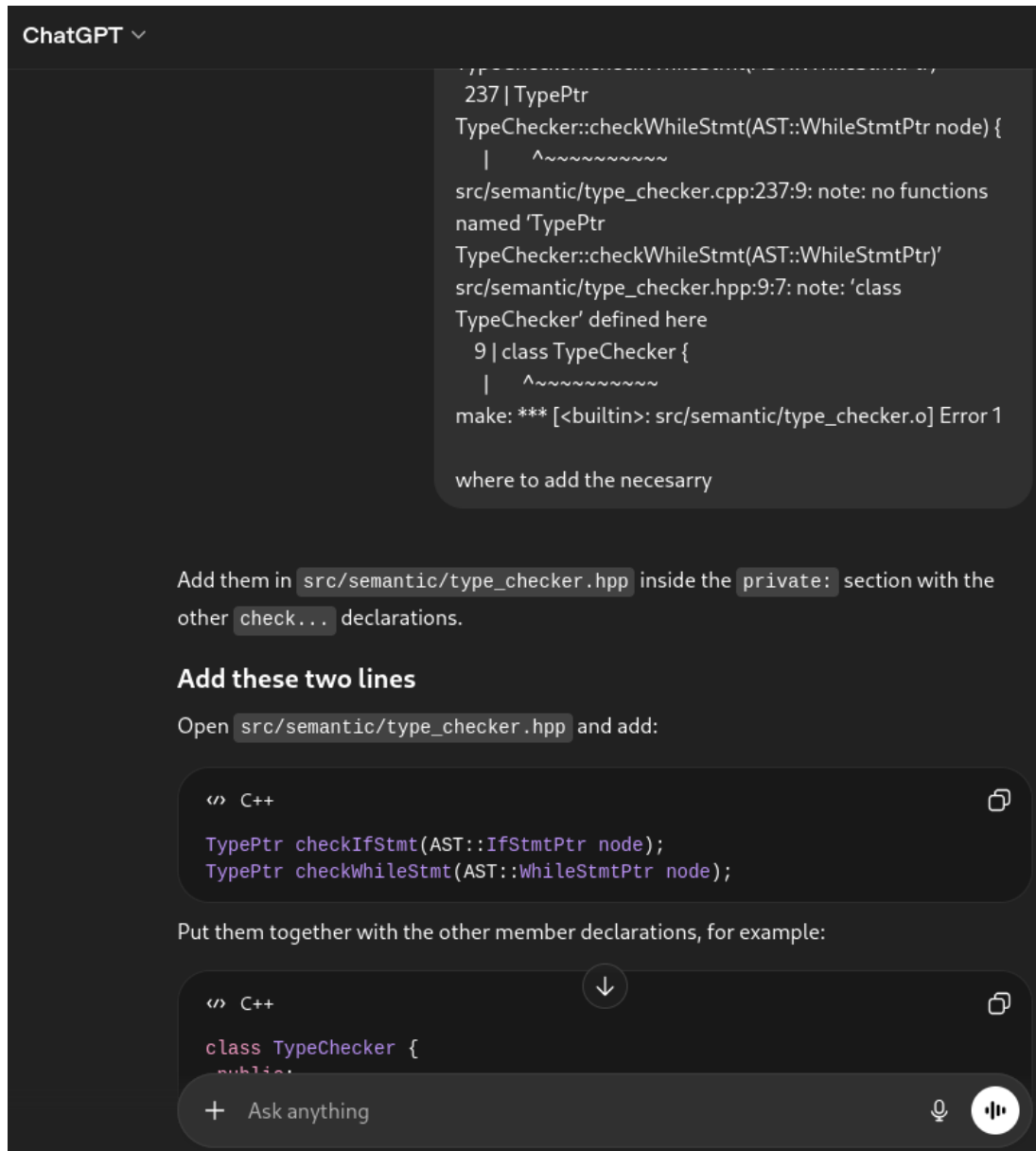


```
auto it = scope_stack.back().find(name);
```

+ Ask anything



For some code error with minor tweaks, I sent the error along with the corresponding files for it to search the error with explanation :



During testing phase :

1. I'm making sure I did not miss anything for any failed test cases

ChatGPT ▾

test
File

here are the test file, I think I left some check definition, help me identify what I miss

I'm going to inspect the failing test contents directly so we can stop guessing and map each wrong answer to the exact semantic rule your checker is missing.

Thought for 2m 24s >

Yes — the test results help a lot, and they show **the last fix was not the root issue**.

Your current `type_checker.cpp` only dispatches a limited set of AST node types, and the failure pattern shows you still are not traversing some nodes correctly and still have a couple of semantic rules modeled too simply. You can see the current dispatch list in your file, and it only includes `CompUnit`, `FuncDef`, `VarDecl`, `Block`, `AssignStmt`, `ReturnStmt`, `LVal`, `IntConst`, `FuncCall`, `UnaryExp`, `BinaryExp`, `IfStmt`, and `WhileStmt`. `type_checker` The test report still has many valid-program failures and many semantic-category failures, which means the checker is still missing AST coverage and a few rule details. `Pasted text`

Here are the next tweaks that matter most.

1. Add `InitVal` to the dispatcher

A lot of your initializer tests are still failing, and right now `check()` does not dispatch `InitVal` at all. That means when a scalar initializer or nested initializer is visited, it can still fall into the unknown-node path. Your current dispatch list does not include it. `type_checker`

+ Ask anything

Add this in `check(AST::NodePtr node) :`

C++

CHECK_NODE(InitVal)

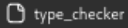
Add this declaration in `type_checker.hpp :`

C++

TypePtr checkInitVal(AST::InitValPtr node);

2. Sometimes, ChatGPT also fixes the major problem from the current code

4. Fix array-parameter compatibility in function calls

Your failures on `funcparam`, `unmatch_call_array1`, and `unmatch_call_array2` strongly suggest your current call checking is too strict for array parameters. Right now `checkFuncCall` uses plain `compatible()` for all arguments. 

But your function parameters are built from `dims_after_first`, which means formal array parameters omit the first dimension, while actual array variables still carry all dimensions. That mismatch can make correct programs fail.

So in `checkFuncCall`, replace this:

```
⟳ C++  
  
if (!func_type->param_types[i]->compatible(actual)) {  
    std::exit(4);  
}
```

with:

```
⟳ C++  
  
auto formal_arr = std::dynamic_pointer_cast<ArrayType>(func_type->param_ty  
auto actual_arr = std::dynamic_pointer_cast<ArrayType>(actual);  
  
if (formal_arr && actual_arr) {  
    if (!formal_arr->element_type->compatible(actual_arr->element_type)) {  
        std::exit(4);  
    }  
}  
  
// formal dims should match actual dims after dropping the first dimensi
```

+ Ask anything

I spent a lot of time for the `array_init_excess`, so I had to ask ChatGPT for the implementation fixes:

 File **array_init_not_list.sy**
File

looks like any array excessive test code still fails, help me point out the potential related problem

You're very close now. The remaining failures split into just three root causes: array initializer form/counting, array-parameter representation, and one leftover incompatible-conversion path that your current initializer logic is still letting through.

Thought for 24s >

Great — 51/59 means the core checker is working. The 8 failures now come from just **3 specific issues**:

1. **array initializer form/counting**
2. **array parameter representation**
3. **one remaining incompatible-conversion path**

Your latest failed tests show exactly that:

- `array_init_excess1..4` expect `3` but your compiler returns `0`
- `array_init_not_list` expects `12` but your compiler returns `0`
- `unmatch_call_array1/2` expect `4` but you return `6` from inside the function body
- `type_incompatible` expects `4` but you return `0`  Pasted text

Here's how to fix each one.



1. Fix array initializer form and excess counting

Why your current code fails

+ Ask anything



Here's how to fix each one.

1. Fix array initializer form and excess counting

Why your current code fails

Right now, for arrays, you only do:

- "is it an `InitVal`?"
- "is top-level element count > total capacity?"

That misses cases like:

- array initialized by scalar form inside `InitVal <expr>` → should be 12
- nested initializers whose **flattened scalar count** exceeds capacity → should be 3

That is exactly why `array_init_not_list` and `array_init_excess1..4` are still wrong.

 Pasted text

What to add in `type_checker.hpp`

Add these declarations in `private:`:

 C++



```
int count_init_elements(AST::InitValPtr node);  
bool is_list_init(AST::InitValPtr node);
```



Add these helper functions in `type_checker.cpp`

+ Ask anything

